Related work & useful tools and technologies

1. **NVIDIA Garak (LLM Vulnerability Scanner)**

Garak e un scanner de vulnerabilități care seamănă cu Nmap sau Metasploit, dar este conceput pentru LLM-uri. Rolul său este să forțeze un LLM să eșueze pentru a putea testa limitele de Securitate (hallucinații, scurgeri de date, etc)

- **Date**: baze de date care conțin tehnici de read-teaming, prompt injection, data leakage și jailbreaks, structurate pe categoriile OWASP Top 10 LLM.
- **Algoritmi**: agenți AI și generatoare care construiesc și trimit prompturi malițioase pentru a evalua punctele slabe ale modelului testat.
- **Performanțe**: rularea automata a sute de teste și descoperirea unor vulnerabilități pe care testele simple de obicei le ratează
- **Librării/Tehnologii & Git:** Python; open-source GitHub (https://github.com/NVIDIA/garak)

Garak oferă seturi de date care ne vor ajuta la antrenare și tuning atunci când vom înlocui predicția hardcodată din AI-ul din proiectul nostru. De asemenea, ne-am inspirat de aici pentru ideea de a renunța la o interfață grafică, fiind probabil mai util ca proiectul nostrum să fie rulat din linia de comandă.

2. **LLM Guard (Protect AI)**

LLM Guard este un toolkit creat pentru apărarea aplicațiilor, fiind ca un firewall între utilizator și LLM.

- **Date**: seturi de date care conțin Personal Identifiable Information, tipare de toxicitate și semnături de prompt-injection, folosite pentru a sanitiza inputul și outputul
- **Algoritmi**: Modele de clasificare care analizează și filtrează datele în timp real
- **Performanțe**: Reducerea riscului de scurgere de informații sensibile + blocarea prompt-injectionurilor, având latența de câteva milisecunde.
- **Librării/Tehnologii & Git:** Python; open-source GitHub (https://github.com/protectai/llm-guard)

Mecanismele de filtrare din LLM Guard ne vor ajuta atunci când AI-ul va oferi sugestiile de remediere pentru vulnerabilitățile identificate.

3. **NeMo Guardrails (NVIDIA)**

NeMo Guardrails este un instrument care le permite dezvoltatorilor să impună reguli ("guardrails") LLM-urilor, asigurându-se că nu deviază de la subiect și nu execută acțiuni nesigure.

- **Date**: dialoguri și seturi de reguli concepute pentru a menține agenții doar pe topicuri autorizate (deci fără politică, medical advice, etc)
- **Algoritmi**: Colang (specific pentru modelarea dialogului) + Execution & Output guardrails, care evaluează deciziile LLM-ului.
- **Performanțe**: Prevenirea execuțiilor neautorizate de cod sau de instrumente + blocarea de system prompt leakages
- **Librării/Tehnologii & Git:** Python, cu integrare nativă pentru LangChain; open-source GitHub (https://github.com/NVIDIA-NeMo/Guardrails)

Modul în care NeMo interceptează și controlează fluxul datelor ne ajută să înțelegem cum arată un cod sigur pentru agenți. Acest lucru ne va oferi un model de referință pe care scannerul nostru să îl poată recunoaște.

4. **Promptfoo (LLM Evaluation Framework)**

Promptfoo este un CLI și librărie pentru evaluarea calității și securității aplicațiilor bazate pe LLM-uri, testând prin read-teamming modelele înainte de lansarea în producție.

- **Date**: Seturi de teste personalizate și matrici de evaluare. Acestea descriu comportamentul așteptat de la model în diferite scenarii de atac sau eroare
- **Algoritmi**: se bazează pe conceptul de LLM-as-a-judge, unde un model evaluează și notează răspunsurile unui model mai slab.
- **Performanțe**: Automatizarea testării, rulând mai multe prompturi în același timp și generând rapid matrici cantitative de tip pass/fail
- **Librării/Tehnologii & Git:** Node.js / TypeScript și funcționează din linia de comandă. open-source GitHub (https://github.com/promptfoo/promptfoo)

În faza finală de testare a AI-ului nostru ne putem folosi de metoda llm-as-judge, automatizând comparația dintre AI-ul intenționat vulneravil și versiunea corectă.